

LAB 3 – SOFTWARE ARCHITECTURE

Peer-to-Peer Carpooling & Expense Splitter

Component Diagram + Architecture Justification

Name VISMAYA G	
SRN : PES1UG24AM326	
Section : F	

1. Architecture Selection

The selected architecture for the Peer-to-Peer Carpooling & Expense Splitter is Microservices Architecture. The system is divided into independent services, with each service responsible for a specific business function and communicating through defined interfaces/APIs.

2. Architecture Comparison

Architecture	Main Idea	Suitability	Decision
Layered	System organized into presentation, business and data layers.	Simple and maintainable, but tightly couples many functions within the application.	Not selected
Client-Server	Clients request services/resources from a central server.	Simple for a small centralized system, but the central server can become a bottleneck.	Not selected
Microservices	Application split into independent services communicating through APIs.	Fits separate user, ride, matching, payment and rating functions and allows independent scaling.	Selected

3. Component Diagram

UML Component Diagram
Peer-to-Peer Carpooling & Expense Splitter

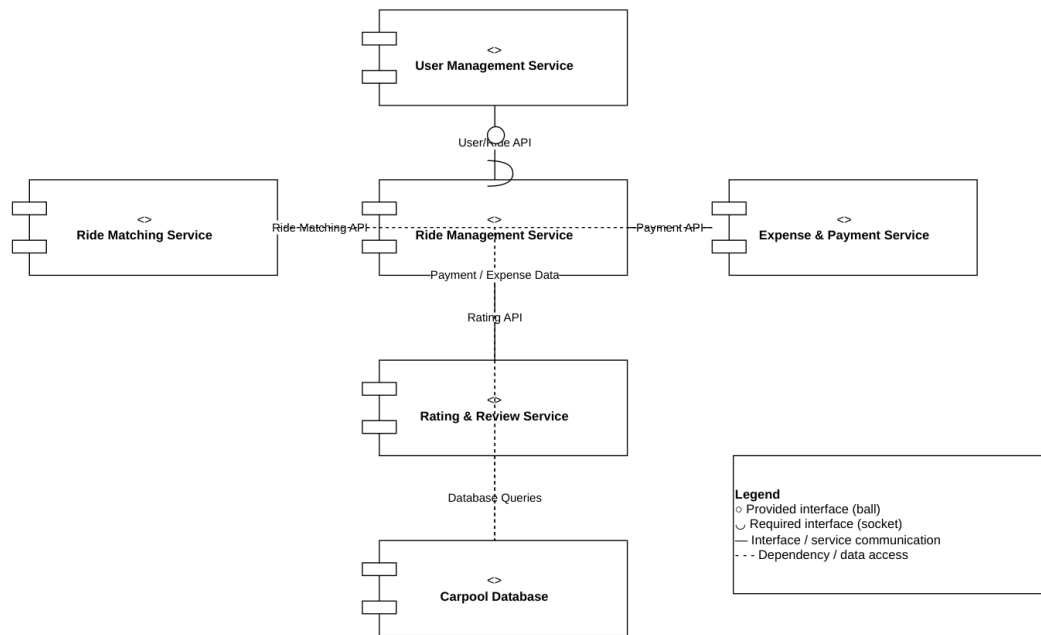


Figure 1: UML Component Diagram for the Peer-to-Peer Carpooling & Expense Splitter

4. Components and Responsibilities

User Management Service: Handles user registration/login and user-related information.

Ride Management Service: Manages creation, publishing, requesting and updating rides.

Ride Matching Service: Searches for compatible rides and supports route compatibility checks.

Expense & Payment Service: Handles fuel-cost sharing and payment/expense-related operations.

Rating & Review Service: Manages post-trip ratings and reviews between users.

Carpool Database: Stores user, ride, route, rating and payment/expense data required by the services.

5. Interfaces and Interactions

Interface / Interaction	Purpose
User/Ride API	Connects User Management Service and Ride Management Service.
Ride Matching API	Allows Ride Management Service to communicate with the Ride Matching Service.
Payment API	Connects Ride Management Service with the Expense & Payment Service.
Rating API	Connects Ride Management Service with the Rating & Review Service.
Database Queries	Allows services to retrieve and store persistent application data.
Payment / Expense Data	Stores payment and expense-related information in the Carpool Database.
Route / Ride Data	Supports storage and retrieval of route and ride information used for matching.

6. Justification for Selecting Microservices

Reason 1 – Modularity and Maintainability

Microservices separate the major functions of the carpooling application. For example, improvements to the Ride Matching Service can be made without changing the Expense & Payment or Rating & Review services. This makes the system easier to maintain, test and extend.

Reason 2 – Independent Scalability

Different services can experience different workloads. Ride search and matching may receive many requests at the same time, while rating operations may receive fewer requests. Microservices allow heavily used services to be scaled independently instead of scaling the entire application.

7. Security Advantage

Security-sensitive functions can be isolated into dedicated services. User authentication and account information can be handled by the User Management Service, while payment and expense information can be handled by the Expense & Payment Service. Access between services can be controlled through authenticated and authorized APIs, reducing unnecessary exposure of sensitive data.

8. Performance Benefit

The Ride Matching Service can be optimized and scaled independently because ride searching and compatibility checking are expected to be frequent operations. Independent service scaling and focused processing can reduce response time and help the application handle multiple concurrent ride searches efficiently.

9. Conclusion

Microservices Architecture is suitable for the Peer-to-Peer Carpooling & Expense Splitter because it provides modularity, independent scalability, service-level security isolation and performance optimization. The architecture also makes it easier to add future services without redesigning the complete application.